Using Planktos

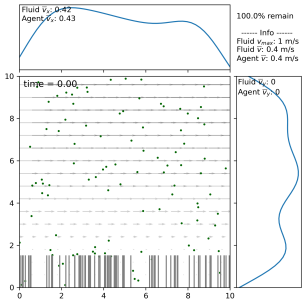
Agent-based transport and behavior in fluid environments

Laura A. Miller

Department of Mathematics
University of Arizona

IBFE Tutorials

AI tools were used to assist with formatting, organization, and clarification of the instructor's original notes.



Planktos documentation image `docs/_static/basic_ex_2d_agents.png`

(commit `a1c2128`, GPLv3).

What you will be able to build

Core workflow

- create a 2D or 3D `Environment`;
- specify outer-domain boundary conditions;
- generate or import a velocity field and mesh;
- initialize one or more vectorized `Swarms`;
- advance, inspect, plot, animate, and export.

Research extensions

- encode deterministic, stochastic, or social behavior;
- choose sliding, sticky, or disabled mesh collisions;
- compute KDE and finite-time Lyapunov exponent fields;
- reproduce a current Vicsek-model example.

Every code fragment targets **Planktos v1.0.3**, repository commit `a1c2128` (2026-08-19). Unless a slide explicitly continues prior state, treat each fragment as a separate pattern in a fresh environment.

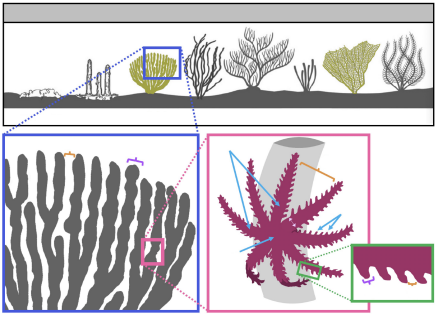
Modeling scope: agents respond to a prescribed flow

Planktos is designed for point-particle organisms or particles whose feedback on the surrounding flow is negligible. A useful template is

$$d\boldsymbol{X} = \big[\boldsymbol{u}(\boldsymbol{X}, t) + \boldsymbol{b}(\boldsymbol{X}, t; \theta)\big]\, dt + \boldsymbol{\sigma}\, d\boldsymbol{W}_t,$$

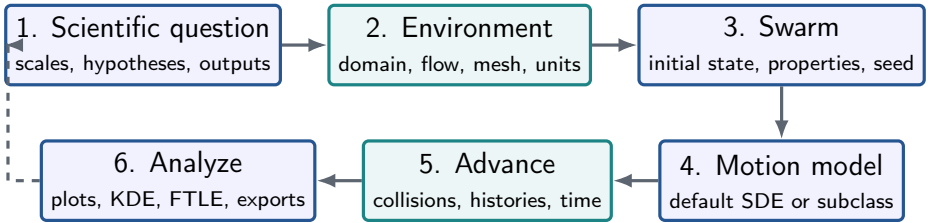
- $\boldsymbol{u}$: prescribed CFD or analytical velocity field;
- $\boldsymbol{b}$: swimming, settling, taxis, or social drift;
- $\boldsymbol{\sigma}$: stochastic variability, constant in space (it may still differ per agent);
- boundaries and immersed meshes constrain the proposed step.

A spatially varying $\boldsymbol{\sigma}(\boldsymbol{X})$ needs a Milstein-type scheme; the built-in Euler–Maruyama step is only order 0.5 in that case and does not support it directly. **One-way coupling.** Agent



Biological scales and gorgonian morphology: Hamlet et al. (2023), Fig. 1 (adapted from Kupfner Johnson & Hallock, 2020), CC BY 4.0.

The end-to-end workflow



Treat plotting and diagnostics as part of the simulation loop: inspect early, then scale up.

Install from the cloned repository

Clone the official repository, then make an isolated environment inside it.

```
git clone https://github.com/mountaindust/Planktos.git
cd Planktos
python -m venv .venv

# PowerShell
.\.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate

python -m pip install --upgrade pip
python -m pip install -e .
python -c "import planktos; print(planktos.__version__)"
```

Expected version for this deck: `1.0.3`.

Editable install

`-e` points the environment at the clone, so a later `git pull` updates the import without reinstalling.

Smoke test

Run `python examples/basic_ex_2d.py`; close each interactive plot to continue.

Adapted from the repository `README.md` Quickstart.

Dependencies: what is installed, and what is optional

Declared core	Role
Python ≥ 3.8	runtime (development currently uses newer Python)
NumPy ≥ 1.19	vectorized agent and field arrays
SciPy $\geq 1.10.1$	interpolation, KDE, ODE integration
Matplotlib ≥ 3.0	plots and animations
pandas	per-agent properties
VTK/PyVista	≥ 9.2/ ≥ 0.44; CFD/mesh I/O and VTK output

Versions and extras are from `setup.cfg` at commit `a1c2128`.

Optional extras

```
python -m pip install -e
      ".[STL,netCDF,test]"
```

- `numpy-stl`: STL input;
- `netCDF4`: NetCDF input;
- `pytest`: test suite.

VTK installation

If binary resolution becomes difficult, the project documentation recommends a clean conda-forge/Miniforge environment.

FFmpeg is needed only when saving animations

```
# Verify that the executable is on PATH
ffmpeg -version

# PowerShell can also locate it explicitly
Get-Command ffmpeg
```

Without FFmpeg

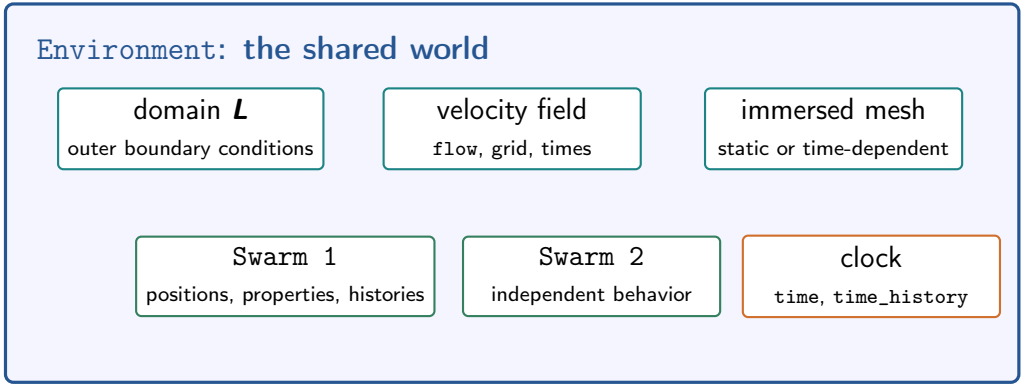
Simulations, static plots, and interactive animations still work. A filename passed to `plot_all` triggers the external movie writer.

If it is not found

1. install FFmpeg with the platform's package manager or official installer;
2. add its executable directory to `PATH`;
3. restart the terminal or IDE;
4. rerun `ffmpeg -version` in the same environment that runs Python.

For long simulations, save every k-th state as a frame; rendering every time step is rarely useful.

Two objects organize almost everything



- A swarm is a vectorized group: agents are rows of NumPy arrays, not separate Python objects.
- All swarms in one environment sample the same flow and mesh.

Create the first environment

```
import planktos

env = planktos.Environment(
    Lx=20, Ly=5,
    x_bndry="zero",
    y_bndry=["noflux", "noflux"],
    rho=1000, mu=1.0,
    units="m")

print(env.L)        # [20, 5]
print(env.nu)       # mu/rho = 0.001
print(env.time)     # 0.0
```

Defaults worth knowing

- 10×10 two-dimensional domain;
- zero velocity everywhere;
- `zero` boundaries in x, y;
- no swarms or immersed mesh;
- `time = 0.0`.

Units are metadata

`units` does not rescale data automatically. Make the flow, mesh, agent parameters, gravity, and time convention consistent yourself.

Outer-domain boundary conditions

Setting	What happens to an agent crossing the box	Typical use
`zero`	The agent is masked and no longer updated or plotted.	open/outflow boundary
`noflux`	The proposed displacement through the wall is projected along it.	solid outer wall
`periodic`	The agent exits one side and re-enters on the opposite side.	repeating domain

Each axis accepts one string for both sides or a two-entry sequence:

```
x_bndry=["zero","noflux"].
```

The order is low-side, high-side: left/right, bottom/top, and low/high z.

Do not conflate two layers

These settings govern the rectangular domain. Immersed-mesh collisions are controlled separately by `ib_condition` or `ib_collisions`.

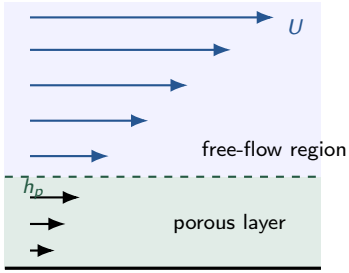
Generate a Brinkman-layer flow

```
env = planktos.Environment(
    Lx=20, Ly=5, rho=1000, mu=1000)

env.set_brinkman_flow(
    alpha=66, h_p=1.5,
    U=1, dpdx=1, res=101)

env.plot_flow()
```

- `alpha`: inverse hydraulic-permeability parameter;
- `h_p`: porous-layer height;
- `U`: velocity at the top of the domain;
- `dpdx`: pressure-gradient parameter;
- `res`: points across the resolved profile.



Code adapted from `examples/basic_ex_2d.py`.

Read the field representation before moving agents

```
print(len(env.flow))
print([a.shape for a in env.flow])
print(env.flow_points)
print(env.flow_times)
print(env.L, env.units)
```

Array convention

- static: `(nx, ny[, nz])`;
- time-dependent: `(nt, nx, ny[, nz])`;
- one array for each velocity component;
- `flow_points[i]` indexes spatial axis i.

Interpolation contract

Planktos uses linear spatial interpolation on a rectilinear grid and cubic splines in time for time-dependent data.

Preflight checks

- component shapes agree;
- coordinate arrays increase and match the axes;
- domain boundaries and mesh coordinates align;
- `flow_times` spans the intended run;
- speeds and units are physically plausible.

Supply a hand-built rectilinear velocity field

```python
import numpy as np
import planktos

x = np.linspace(0, 2, 81)
y = np.linspace(0, 1, 41)
X, Y = np.meshgrid(x, y, indexing="ij")

u = 4*Y*(1-Y)          # streamwise component
v = np.zeros_like(u)

env = planktos.Environment(
    Lx=x[-1], Ly=y[-1], flow=[u, v])
env.flow_points = (x, y)
env.plot_flow()
```

Why `indexing="ij"`?

Axis 0 is x, axis 1 is y. This matches the current Planktos storage convention and avoids a silent transpose.

Time-dependent fields

Stack snapshots on axis 0 and pass `flow_times` to the constructor. The remaining axes stay $(x, y[, z])$.

Coordinates

Set `flow_points` explicitly when the coordinates are not the constructor's uniform grid.

Convention cross-checked against current flow-generation and analysis tests.

Add a vectorized swarm

```
# Convenience method: automatically uses env
s = env.add_swarm(
    swarm_size=200,
    init="random", seed=2026,
    name="larvae", color="darkgreen")

# Equivalent explicit construction
s2 = planktos.Swarm(
    swarm_size=50, envir=env,
    init=(0.1, 0.5), seed=7)

print(s.N, s.positions.shape)
print(len(env.swarms))
```

State layout

- `positions`: masked $N \times d$ array;
- `velocities`: $N \times d$;
- `accelerations`: $N \times d$;
- `props`: per-agent DataFrame;
- `shared_props`: shared dictionary.

Vectorization is the design choice: write array operations over agents whenever possible.

Choose initial positions deliberately

```
# Uniform random over the domain
s_random = env.add_swarm(500, init="random", seed=1)

# One shared starting point
s_point = env.add_swarm(100, init=(0.15, 0.50), seed=2)

# Explicit N-by-d array
rng = np.random.default_rng(3)
xy = np.column_stack((
    rng.uniform(0.05, 0.15, 300),
    rng.uniform(0.20, 0.80, 300)))
s_patch = env.add_swarm(init=xy, seed=3)

# Grid, optionally excluding closed mesh interiors
s_grid = env.add_swarm(
    init="grid", grid_dim=(80, 40), testdir="x1")
```

Interpretation

- `random`: a uniform domain sample;
- point: a release cohort;
- $N \times d$ array: measured or designed initial condition;
- `grid`: transport-map or FTLE seed lattice.

Grid masking

`testdir` is meaningful only when a closed immersed mesh is loaded and the chosen ray direction gives a robust inside/outside test.

Shared properties, individual variation, and seeds

```python
import numpy as np

# Same for every agent
s.shared_props["mu"] = np.array([0.0, 0.0])
s.shared_props["cov"] = np.eye(2)*(0.01**2)
s.shared_props["diam"] = 1.0e-3

# One value per agent
rng = np.random.default_rng(2026)
s.props["swim_speed"] = rng.lognormal(
    mean=np.log(0.02), sigma=0.15, size=s.N)

# Uniform access: scalar/array as appropriate
speed = s.get_prop("swim_speed")
diam = s.get_prop("diam")
```

The two random streams

- `seed=` initializes `s.rndState`, used by built-in stochastic motion.
- A separate NumPy generator should seed any user-created initial conditions or properties.

Covariance units

The default SDE uses step covariance $\mathrm{cov}\,dt$. Thus cov has units of length^2/time, not length^2 per step.

The default model is Euler–Maruyama drift–diffusion

For each active agent, the default `apply_agent_model` approximates

$$d\boldsymbol{X} = [\boldsymbol{u}(\boldsymbol{X}, t) + \boldsymbol{\mu}]\, dt + \boldsymbol{\sigma}\, d\boldsymbol{W}_t, \qquad \mathrm{cov} = \boldsymbol{\sigma}\boldsymbol{\sigma}^T = 2\boldsymbol{D}.$$

One step is

$$\boldsymbol{X}^{n+1,*} = \boldsymbol{X}^n + [\boldsymbol{u}(\boldsymbol{X}^n, t^n) + \boldsymbol{\mu}]\, \Delta t + \mathcal{N}(\boldsymbol{0}, \mathrm{cov}\, \Delta t),$$

followed by outer-boundary and immersed-mesh handling.

Deterministic tracer

For spatial dimension (d), set `mu = np.zeros(d)` and `cov = np.zeros((d,d))`; then motion is forward Euler using the local fluid velocity.

Time-step audit

Check both temporal convergence and collision resolution. A step can be statistically reasonable yet too large to resolve a narrow obstacle.

Advance the swarm and read its histories

```
import numpy as np

dt = 0.1
for _ in range(round(24.0/dt)):
    s.move(dt, silent=True)

print(env.time)
print(len(env.time_history))
print(len(s.full_pos_history))

# Current diagnostics
mask = np.ma.getmaskarray(s.positions)
active = ~np.all(mask, axis=1)
print(active.sum(), "agents remain active")
print(s.velocities.mean(axis=0))
```

What `move` manages

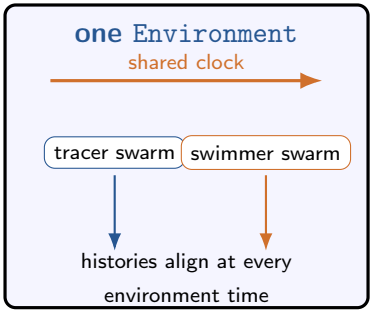
1. calls `apply_agent_model(dt)`;
2. stores prior position/velocity/property state;
3. estimates velocity and acceleration;
4. applies boundaries and mesh collisions;
5. calls `after_move(dt)`;
6. advances environment time.

History note: `full_pos_history` and `full_vel_history` return stored history plus the current state without mutating it.

Move multiple swarms on one shared clock

```
tracers = env.add_swarm(
    500, seed=10, name="tracers", color="navy")
swimmers = env.add_swarm(
    500, seed=11, name="swimmers", color="orange")
swimmers.shared_props["mu"] = np.array([0.02, 0.0])

for _ in range(400):
    env.move_swarms(
        dt=0.05,
        ib_collisions="sliding",
        silent=True)
```



Avoid double time advancement

Do not call `s1.move(dt)` and then `s2.move(dt)` when both should occupy the same time step. Use `env.move_swarms`.

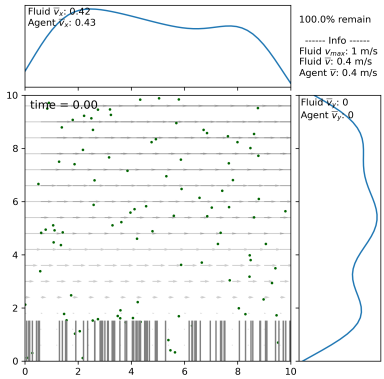
Make an early diagnostic plot

```
# Current state with velocity arrows
s.plot(
    fluid="quiver",
    dist="density",
    circ_rad=0.08,
    plot_heading=False)

# Closest stored state to t=8.0
s.plot(
    t=8.0,
    filename="snapshot.png",
    fluid="vort",
    save_kwargs={"dpi": 200})
```

2D fluid overlays

`fluid="quiver"` shows velocity vectors; `fluid="vort"` shows scalar vorticity. These overlays are 2D-only.



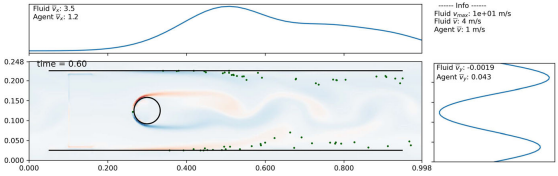
Not the output of the calls above, but a documentation render of the same example: Planktos documentation image `docs/_static/basic_ex_2d_agents.png` (commit `a1c2128`, GPLv3).

KDE turns particles into a distribution diagnostic

```
# Gaussian KDE with Scott's factor
s.plot(dist="density", fluid="vort")

# Covariance-based width
s.plot(dist="cov", fluid="vort")

# User bandwidth factor or a histogram
s.plot(dist=0.35)
s.plot(dist="hist")
```



IB2d cylinder flow, agents, vorticity, and marginal density: Strickland et al. (2022), Fig. 4; educational reproduction.

- Compare concentration shape, tails, and obstacle wakes.
- Report the bandwidth rule with any quantitative comparison.
- Check sensitivity to sample size and boundary truncation.
- A KDE is a smoothed estimator, not a separately solved concentration field.

Animate selectively; export analysis-ready data

```
# Use every fifth stored state
frames = range(0, len(s.full_pos_history), 5)
s.plot_all(
    movie_filename="trial_01.mp4",
    frames=frames, fps=10,
    fluid="vort", plot_heading=False)

# Data products
s.save_data("results", "trial_01")
s.save_pos_to_csv("results/trial_01_positions.csv")
s.save_pos_to_vtk("results", "trial_01")
```

Choose the product by use

- `save_data`: position history plus current kinematics and properties;
- CSV: portable position history;
- VTK: ParaView/VisIt workflow;
- MP4: communication, not raw data.

Cost control

Use `frames` for temporal subsampling and `downsamp` for display-only agent subsampling. Statistics still use the full population.

Calls match current `Swarm.plot_all` and save-method signatures.

Custom behavior: override one hook, not the harness

```python
class MySwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        # Compute proposed N-by-d positions.
        new_positions = ...
        return new_positions
```

Contract

- keep the signature `(self, dt)`;
- return an $N \times d$ array;
- do not assign `positions`, `velocities`, or `accelerations`;
- let `move` record history and apply collisions.

Current contract: `planktos/_swarm.py` and `examples/ex_agent_behavior.py`.

Current versus historical API

The 2022 paper describes a historical override named `get_positions`. The current v1.0.3 repository and examples use `apply_agent_model(self, dt)`.

Use `after_move(dt)` for state changes based on the accepted, collision-corrected endpoint.

Example: fluid advection plus directed swimming

```python
import numpy as np
import planktos

class TargetSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        target = np.array([1.8, 0.5])
        direction = target - self.positions
        norm = np.linalg.norm(direction, axis=1)[:, None]
        direction = direction/np.maximum(norm, 1.0e-12)

        speed = self.get_prop("swim_speed")[:, None]
        velocity = self.get_fluid_drift() + speed*direction
        return self.positions + velocity*dt
```

Pattern adapted from `examples/ex_agent_behavior.py`.

Useful field queries

- `get_fluid_drift()`;
- `get_dudt()`;
- `get_fluid_mag_gradient()`;
- `get_DuDt()`.

Numerical hygiene

Guard normalizations, preserve masks, and call expensive interpolants once per time step rather than once per agent.

Deterministic ODE motion with the RK45 wrapper

```
import planktos
from planktos import motion

class AccurateTracer(planktos.Swarm):
    def apply_agent_model(self, dt):
        ode = motion.tracer_particles(
            self, incl_dvdt=False)
        t0 = self.envir.time
        # positions is a masked array; strip masked
        # (exited) rows before RK45, as calculate_FTLE does
        return motion.RK45(
            ode, t0, self.positions.copy(), t0 + dt,
            rtol=1.0e-6, atol=1.0e-9)
```

Current solver contract

`motion.RK45` wraps SciPy `solve_ivp`; the right-hand side has signature `fun(t, x)` and returns the same 2D shape as `x`.

Two time scales

The internal ODE solver adapts substeps, but Planktos applies boundary and mesh collision handling only after the outer `move(dt)` proposal.

Combine an ODE drift with stochastic motion

```python
import numpy as np
import planktos
from planktos import motion

def social_bias(pos):
    return 0.1*(pos.mean(axis=0) - pos)

def behavior_ode(swarm):
    def rhs(t, x):
        N = x.shape[0]//2
        vel = swarm.get_fluid_drift(t, x[:N])
        vel += social_bias(x[:N])
        return np.concatenate((
            vel, np.zeros((N, x.shape[1]))))
    return rhs

class SocialSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        return motion.Euler_brownian_motion(
            self, dt, ode=behavior_ode(self))
```

Adapted from `examples/ex_ode_gen.py`.

State convention

An ODE passed to `Euler_brownian_motion` receives $2N \times d$: positions followed by velocities. It returns their time derivatives in the same layout.

Fluid drift is explicit

When an `ode` is supplied, include fluid interaction in that ODE if it is part of the model; it is not added automatically there.

Built-in inertial-particle dynamics

Set ρ, μ, ν in the constructor: the property setters in v1.0.3 compute $\mu = \nu/\rho$ (bug, reported upstream).

```
import numpy as np
import planktos
from planktos import motion

zero = np.zeros((3, 3))
env = planktos.Environment(
    Lx=1, Ly=1, rho=1000, nu=1e-6,
    char_L=0.01, U=0.1,
    flow=[zero, zero.copy()])

class InertialSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        ode = motion.inertial_particles(self)
        # positions is a masked array; strip masked
        # (exited) rows before RK45, as calculate_FTLE does
        # NB: move() overwrites velocities with
        # (x_new-x_old)/dt; persist velocity in props
        # for a true second-order model
        state = np.ma.concatenate(
            (self.positions, self.velocities))
        out = motion.RK45(
            ode, self.envir.time, state,
            self.envir.time + dt)
        return out[:self.N]

s = InertialSwarm(envir=env, seed=4)
s.shared_props.update({"R": 2/3, "diam": 1e-4})
```

Required environment data

A velocity field (explicit zeros are valid), `char_L`, `U`, `nu`, and gravity; fluid density unless `R` is supplied.

Required particle data

Density ratio `R` (or particle `rho`) and diameter `diam`.

Model regime

The implementation is the linearized Maxey–Riley form documented for $R/\mathrm{St} \gg 1$; verify that approximation for the particles and flow under study. Here, the displayed parameters give $R/\mathrm{St} \approx 120$.

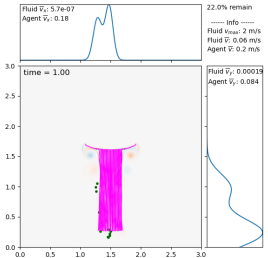
Immersed-mesh collision modes

Mode	Proposed step after a mesh hit
`sliding`	Project motion along the boundary, recursively using remaining displacement.
`sticky`	Stop at the first point of intersection for that time step.
`None`	Ignore immersed meshes.
`'default'`	Use the Swarm's `ib_condition` (default `'sliding'`).

Set a swarm default with `ib_condition`; override per step with

```
s.move(dt, ib_collisions="sticky").
```

`s.ib_collision_idx` is a per-agent array of the last-step element index hit, -1 if none.



Sticky agents and a moving IB2d boundary; Planktos documentation image `docs/_static/ib2d_mvbnd_sticky.png` (commit `a1c2128`, GPLv3).

Geometry restriction

Avoid immersed structures that intersect periodic box boundaries; that interaction is not implemented in the current API.

Make “captured” a persistent biological state

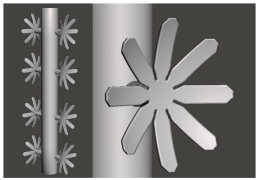
```python
import numpy as np
from planktos import motion

class CaptureSwarm(planktos.Swarm):
    def apply_agent_model(self, dt):
        proposed = motion.Euler_brownian_motion(self, dt)
        held = self.get_prop("captured")[:, None]
        return np.ma.where(held, self.positions, proposed)

    def after_move(self, dt):
        hit = self.ib_collision_idx >= 0
        self.props.loc[hit, "captured"] = True
        self.props.loc[hit, "color"] = "gold"

s = CaptureSwarm(
    envir=env, ib_condition="sticky",
    store_prop_history=True)
s.props["captured"] = np.zeros(s.N, dtype=bool)
s.props["color"] = np.full(s.N, s.shared_props["color"])
```

Pattern adapted from `examples/ex_sticky_seafan_3d.py`.



Cylinder and polyp geometry: Hamlet et al. (2023), Fig. 6a, CC BY 4.0.

Why both hooks?

`after_move` records new contacts after collision correction.

`apply_agent_model` prevents previously captured agents from leaving on later steps.

Import an IB2d velocity field and 2D mesh

```
env = planktos.Environment()

# From the repository root, first follow
# examples/ib2d_data/README.txt to download the data.
env.read_IB2d_fluid_data(
    "examples/ib2d_data",
    dt=5.0e-5, print_dump=1000)

env.read_IB2d_mesh_data(
    "examples/ib2d_data/channel.vertex",
    method="proximity")

env.plot_envir()
s = planktos.Swarm(
    envir=env, init=(0.1*env.L[0], 0.5*env.L[1]),
    seed=1)
```

Time mapping

Successive printed flow snapshots are separated by

$$\Delta t_{\text{data}} = \texttt{dt} \times \texttt{print_dump} = 0.05.$$

Mesh association

`proximity` connects nearby vertices. It can infer resolution from loaded flow or accept `res=` explicitly.

Inspect connectivity. Crossing segments without a shared vertex can create collision leaks. Always call `plot_envir()`.

Adapted from `examples/ex_ib2d_ibmesh.py`; the external dataset is linked from `examples/ib2d_data/README.txt`.

Import matrix: CFD fields and 3D geometry

Source	Current call pattern	Audit item
IBAMR 3D VTK	`env.read_IBAMR3d_vtk_data(path, d_start=0, d_finish=None, vel_conv=None, grid_conv=None)`	dump range; spatial and velocity conversions
COMSOL VTU	`env.read_comsol_vtu_data(file, vel_conv=None, grid_conv=None)`	cell-centered versus nodal grid; units
NetCDF	`env.load_NetCDF(file)` then `env.read_NetCDF_flow("u","v", time_name="time", dim_reorder=...)`	variable names, dimension order, component axis, calendar/time units
STL mesh	`env.read_stl_mesh_data(file, unit_conv=None)`	mesh and flow share the same coordinate frame
IB2d mesh	`env.read_IB2d_mesh_data(file, method="adjacent"|"proximity"|"hull")`	breaks/additional edges; closed geometry

The loader may reset `env.L` to match the data. Read back `L`, `flow_points`, `flow_times`, `fluid_domain_LLC`, and units after every import.

Signatures audited against `planktos/_environment.py`, v1.0.3.

Align flow, mesh, and physical units before release

```
# For appropriate cell-centered COMSOL data,
# regrid immediately after the flow import.
# Match these flags to the data's actual BCs;
# this example treats all axes as nonperiodic.
env.center_cell_regrid(
    periodic_dim=(False, False, False))

# Only then load the immersed geometry.
env.read_stl_mesh_data("geometry.stl")

print("domain:", env.L, env.units)
print("flow origin:", env.fluid_domain_LLC)
print("flow axes:", [p[[0, -1]] for p in env.flow_points])
print("mesh shape:", None if env.ibmesh is None
      else env.ibmesh.shape)

env.plot_envir()
env.plot_flow()
```

One consistent conversion ledger

Record conversions for length, velocity, time, density, viscosity, gravity, particle diameter, and mesh coordinates. Prefer converting on import with `vel_conv`, `grid_conv`, or `unit_conv`.

Visual mesh audit

Look for crossing edges, missing closure, duplicated surfaces, mesh outside the flow domain, and unintended contact with a periodic face.

FTLE reveals finite-time stretching of a motion model

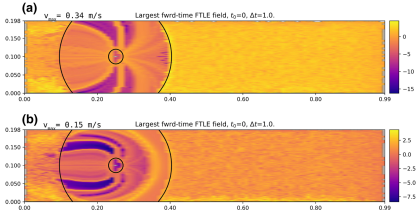
Let $\tau = T - t_0 > 0$. For the flow map $\mathbf{\Phi}_{t_0}^{T}$, the largest forward-time exponent is

$$\sigma_{t_0}^{\tau}(\boldsymbol{x}_0) = \frac{1}{\tau} \log \sqrt{\lambda_{\max}[(D\mathbf{\Phi})^T D\mathbf{\Phi}]}.$$

```
env.calculate_FTLE(
    grid_dim=(512, 128),
    t0=0, T=0.1, dt=0.001)
env.plot_2D_FTLE()

# Backward from physical time 0.1 to 0.0:
# current API uses T as the forward pseudo-time
      endpoint.
env.calculate_FTLE(
    (512, 128), t0=0.1, T=0.2,
    dt=0.001, backward=True)
```

Calls adapted from `examples/ex_produce_ftle_2d.py`.



Agent-based forward-time FTLE comparison: Strickland et al. (2022), Fig. 5; educational reproduction.

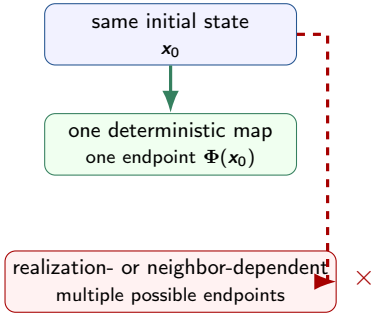
Two API distinctions

- The current call requires `T > t0`; the elapsed interval is `T-t0`. With `backward=True`, physical time ends at `2*t0-T`.
- `FTLE_smallest` is a forward-map contraction exponent, not backward-time FTLE.

When an agent-based FTLE field is meaningful

The `swrm=` route is appropriate when the seeded agents define one reproducible finite-time map:

- the motion rule is deterministic;
- agents use the same rule and parameter set;
- each trajectory acts independently of the other seeded agents;
- the map is resolved well enough in time and on the initial-condition grid.



Not an ordinary FTLE map

Stochastic realizations or social rules that depend on the seeded population do not define a single deterministic particle map. Use ensemble dispersion, encounter, or density diagnostics instead.

Qualification follows Strickland et al. (2022), Sections 2.1 and 4.2.

FTLE is a property of the chosen finite-time map, not of "complex-looking" trajectories in general.

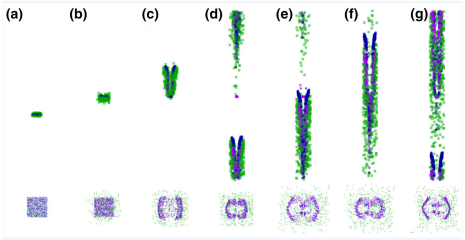
Capstone: Vicsek agents around an immersed cylinder

```
import numpy as np
import planktos

env = planktos.Environment(
    Lx=1, Ly=0.25,
    x_bndry="periodic",
    y_bndry="periodic")
env.read_IB2d_mesh_data(
    "examples/vicsek_mesh/channel.vertex",
    method="proximity", res=0.001953125)

rng = np.random.default_rng(2026)
N = 500
IC = np.empty((N, 2))
IC[:, 0] = rng.uniform(0.075, 0.125, N)
IC[:, 1] = rng.uniform(0.050, 0.200, N)
```

Setup: current `examples/ex_vicsek_model_2d.py`; separate NumPy seed added.



Motivating 3D result: Strickland et al. (2022), Fig. 9; educational reproduction.

Null versus flow experiment

The repository example runs without the large external flow dataset. Load matching IB2d flow to ask how alignment survives an unsteady wake.

Capstone: implement alignment with periodic distances

```
class Vicsek(planktos.Swarm):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.shared_props.update(v=0.02, nu=0.5, r=0.1)
        angle = self.get_prop("nu") \
              * (self.rndState.random(self.N)-0.5)
        self.velocities += self.get_prop("v") \
              * np.column_stack((np.cos(angle), np.sin(angle)))

    def apply_agent_model(self, dt):
        theta = np.empty(self.N)
        L = np.asarray(self.envir.L)
        for i, origin in enumerate(self.positions):
            delta = (self.positions-origin + L/2) % L - L/2
            near = np.linalg.norm(delta, axis=1) < self.get_prop("r")
            theta[i] = np.arctan2(
                self.velocities[near, 1].mean(),
                self.velocities[near, 0].mean())
        theta += self.get_prop("nu")*(self.rndState.random(self.N)-0.5)
        active = self.get_prop("v")*np.column_stack((np.cos(theta),
     np.sin(theta)))
        return self.positions + (active + self.get_fluid_drift())*dt
```

Three model ingredients

1. average neighbor velocity vectors within radius `r`;
2. add angular noise of width `nu`;
3. add self-propulsion speed `v` and local fluid drift.

Periodic metric

The minimum-image displacement is essential: agents near opposite faces can still be neighbors. Note that the minimum-image wrap assumes periodic boundaries in both axes.

Condensed from the current repository Vicsek example; parameter names and hook are unchanged.

Run the capstone, then make the result reproducible

```
s = Vicsek(
    swarm_size=N, envir=env, init=IC, seed=2026)

collision_events = 0
for _ in range(72):       # 18 s at dt=0.25 s
    s.move(0.25, silent=True)
    collision_events += np.count_nonzero(
        s.ib_collision_idx >= 0)

print("agent-step collisions:", collision_events)

s.plot_all(
    movie_filename="vicsek_null.mp4",
    fps=4, fluid="vort")
```

Capstone comparisons

Repeat with a passive swarm, multiple nu/r values, and at least two smaller dt. Compare polarization, retention, agent-step collisions, and downstream spread.

Save with every result

- Planktos version/commit and Python/package environment;
- flow and mesh provenance plus checksums;
- both random seeds and all model parameters;
- time step, duration, collision mode, and active-agent count;
- code, raw histories, and plotting settings.

Failure-state diagnostic

If a collision step fails, current Planktos marks `env.time=None`. Inspect the partial state; then restore consistently from histories or restart from a checkpoint.

Resources and source trail

Start with the current software

- Planktos repository and README;
- repository `docs/` and runnable `examples/`;
- `examples/ib2d_data/README.txt` for the external IB2d dataset;
- this tutorial was audited against v1.0.3, commit `a1c2128` (2026-08-19).

Record the exact software commit with every scientific result: current interfaces can differ from those shown in the 2022 paper.

Primary scientific sources

Strickland et al. (2022), *Bulletin of Mathematical Biology* 84:72

Software architecture, agent models, collision handling, visualization, FTLE, and examples.

Hamlet et al. (2023), *Journal of Experimental Biology* 226, jeb244520

Gorgonian multiscale flow and capture application; article and reproduced figures are CC BY 4.0.